Experiment No. 5

Practical Assignment: Building an Automated CI/CD Pipeline with Jenkins

---

Course: Introduction to DevOps / Cloud Computing

Topic: Continuous Integration & Continuous Deployment (CI/CD) with Jenkins

Level: Beginner to Intermediate

Course Teacher: Mr. Ajit B. Patil

Assignment Objective

By completing this assignment, you will:

1. Connect a source code repository (GitHub) with Jenkins.
2. Write a declarative Jenkinsfile (Pipeline-as-Code).
3. Automate **Build**, **Test**, and **Artifact Archiving** stages.
4. Set up an automated **Build Trigger**.

🛠️ **Prerequisites**

- A running Jenkins instance (Local localhost:8080 or Cloud-hosted).
- A free GitHub account. Git installed on your local machine.

📌 Task Instructions

Task 1: GitHub Repository Setup

1. Create a public repository on GitHub named

   jenkins-student-assignment.

2. Clone the repository to your local machine:

   git clone https://github.com/<your-username>/jenkins-student-assignment.git

   cd jenkins-student-assignment

3. Create a simple sample application file named index.js with a basic hello world message and commit it to your repository.

Task 2: Write a Declarative Pipeline (Jenkinsfile)

Inside your project folder, create a file named Jenkinsfile (no file extension). Write a declarative script that satisfies the following requirements:

- **Stage 1 (Build):** Print a console log saying "Compiling application...".
- **Stage 2 (Test):** Simulate tests by creating an empty test report or printing "Running unit tests... Pass!".
- **Stage 3 (Package):** Generate a text file named build-info.txt containing the current build number and timestamp.
- **Post Action:** Print "Build successful! Ready for release." on success.

Hint: Use the following template as a starting point:

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Compiling application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Running unit tests...'
            }
        }
        stage('Package') {
            steps {
                // Write code here to generate build-info.txt
                sh 'echo "Build executed on $(date)" > build-info.txt'
            }
        }
    }

    post {
        success {
            echo 'Build successful! Ready for release.'
        }
    }
}
```

Commit and push the Jenkinsfile to your main branch on GitHub.

Task 3: Configure Pipeline Job in Jenkins

1. Log into your Jenkins dashboard.
2. Click **New Item** → Name it Assignment-Pipeline \rightarrow Select **Pipeline** → Click **OK**.
3. Scroll down to the **Pipeline** section:
    - **Definition:** Select *Pipeline script from SCM*.
    - **SCM:** Select *Git*.
    - **Repository URL:** Paste your public GitHub repository URL.
    - **Branch Specifier:** Set to */main or */master depending on your default branch.
    - **Script Path:** Ensure it is set to Jenkinsfile.
4. Click **Save**.
5. Click **Build Now** to verify your setup.

Task 4: Enable Automated Triggers

1. Go to your job's **Configure** page.
2. Under **Build Triggers**, select **Poll SCM**.
3. Set the schedule to H/2 * * * * *(Polls GitHub for changes every 2 minutes)*.
4. Modify your app.py file locally, make a new Git commit, and push it to GitHub.
5. Wait up to 2 minutes and verify that **Jenkins automatically triggers Build #2** without human interaction.

Deliverables / Submission Checklist

Students must submit a PDF report containing:

- **Screenshot 1:** GitHub repository displaying Jenkinsfile and application code.
- **Screenshot 2:** Stage View in Jenkins showing all pipeline stages green (SUCCESS).
- **Screenshot 3:** Console Output showing the successful run of Build #2 triggered by Poll SCM.
- **Code:** The raw code of your Jenkinsfile.